

Lesson 11: Fetching from APIs

Apps need data from the internet

Most real apps show data that lives on a server: news articles, product lists, weather, scores. Your app asks the server for this data through an API (Application Programming Interface) -- a set of web addresses that give back data, usually as JSON.

In this lesson you will fetch real data from a public test API and show it on screen.

Adding the http package

Flutter does not include networking by default. You add it as a package -- ready-made code written by others. Open pubspec.yaml and add one line:

```
dependencies:  
  flutter:  
    sdk: flutter  
  http: ^1.2.0
```

Then run `flutter pub get` in the terminal. Now you can use the `http` package in your code:

```
import 'package:http/http.dart' as http;
```

Packages are one of Flutter's strengths. Need maps, video, or payments? There is probably a package for it.

Making a GET request

A GET request asks the server: "give me the data at this address." Here is one to a free test API:

```
Future<void> fetchPost() async {  
  final url = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');  
  final response = await http.get(url);  
  
  print(response.statusCode); // 200 means "OK, here is your data"  
  print(response.body);      // the raw JSON text  
}
```

Two things to check in every response:

- `statusCode` -- 200 means success. 404 means "not found." 500 means the server has a problem.
- `body` -- the actual data, as a JSON string.

Reading JSON

JSON looks like this:

```
{
  "userId": 1,
  "id": 1,
  "title": "Hello world",
  "body": "This is the post text."
}
```

It is just text, so you need to turn it into a Dart map with `jsonDecode`:

```
import 'dart:convert';

Future<void> fetchPost() async {
  final url = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');
  final response = await http.get(url);

  if (response.statusCode == 200) {
    final data = jsonDecode(response.body); // String -> Map
    print(data['title']); // "Hello world"
  }
}
```

`jsonDecode` turns the JSON string into a Map you can read like any other Dart map. Always check the status code first -- only read the body when it is 200.

Showing it in the UI

Combine this with the loading pattern from Lesson 10:

```
class PostScreen extends StatefulWidget {
  const PostScreen({super.key});

  @override
  State<PostScreen> createState() => _PostScreenState();
}

class _PostScreenState extends State<PostScreen> {
  String? title; // null = still loading

  @override
  void initState() {
    super.initState();
    loadPost();
  }

  void loadPost() async {
```

```

try {
  final url = Uri.parse('https://jsonplaceholder.typicode.com/posts/1');
  final response = await http.get(url);
  if (response.statusCode == 200) {
    final data = jsonDecode(response.body);
    setState(() {
      title = data['title'];
    });
  }
} catch (e) {
  setState(() {
    title = 'Could not load. Check your internet.';
  });
}

@override
Widget build(BuildContext context) {
  if (title == null) {
    return const Center(child: CircularProgressIndicator());
  }
  return Center(child: Text(title!));
}
}

```

This is the full real-world flow: loading spinner -> network call -> show data (or an error message). You now know how most apps get their content.

Key terms

- API -- web addresses a server offers so apps can ask for data.
- JSON -- a text format for sending data, readable by almost every language.
- http package -- ready-made code for making network requests.
- GET request -- asking a server to give you data.
- statusCode -- a number telling you if the request worked (200 = success).
- jsonDecode -- turns a JSON string into a Dart map.